E-COMMERCE DATA WAREHOUSE

Analytical SQL Project

Documentation Explaining the Analytical Reasoning

Project Type:	Star Schema Data Warehouse
Database:	MySQL 8.0 / SQLite (Python implementation)
Focus:	Advanced Window Functions & Product Recommendations
Dataset:	1,000 synthetic order lines across 2 years

March 2026

Table of Contents

1. Executive Summary
2. Data Warehouse Architecture
3. Dimensional Modeling Approach
4. KPI Framework & Business Logic
5. Advanced Analytical SQL Techniques
6. Product Recommendation System
7. Data Quality & Validation Strategy
8. Implementation Insights
9. Dashboard & Visualization
10. Conclusion

1. Executive Summary

This document provides comprehensive analytical reasoning behind an E-Commerce Data Warehouse project designed to demonstrate advanced SQL analytics, dimensional modeling, and machine learning principles using SQL-based product recommendations.

1.1 Project Objectives

- Design a star schema optimized for analytical queries
- Implement KPI calculations without predefined formulas
- Leverage window functions for complex time-series and ranking analysis
- Build a SQL-driven recommendation engine using collaborative filtering principles
- Create an interactive Streamlit dashboard for data exploration

1.2 Key Deliverables

Deliverable	Description
Star Schema DDL	6 dimension tables + 1 fact table (01_create_dw_schema.sql)
Synthetic Data	730 date records, 120 customers, 140 products, 1,000 transactions
KPI Views	6 business views covering revenue, products, customers (05_kpi_views.sql)
Recommendation Engine	Multi-factor scoring system with 6 signals (06_recommendation_top4.sql)
Dashboard	Interactive Streamlit app with 4 pages (app.py + db.py)

2. Data Warehouse Architecture

2.1 Star Schema Design Rationale

The star schema was chosen for its analytical optimization characteristics:

- **Query Performance:** Denormalized dimensions minimize joins, improving query speed
- **Simplicity:** Business users can easily understand the one-hop join relationships
- **Aggregation Efficiency:** Pre-calculated measures in the fact table enable fast summary queries
- **Flexibility:** New dimensions can be added without restructuring existing tables

2.2 Fact Table: fact_order_line

Grain: One row per product per order (order-line level)

Measure Design Philosophy:

- **Additive Measures:** All monetary amounts (gross_amount, net_amount, profit_amount) are fully additive across all dimensions
- **Pre-calculated Profit:** Storing profit_amount = net_amount - cost_amount avoids repeated calculations
- **Discount Tracking:** Separate discount_amount field enables promotion analysis
- **Degenerate Dimension:** order_id stored directly in fact (no separate order dimension needed)

2.3 Dimension Tables

dim_date — Calendar Intelligence

Purpose: Enable time-based analysis without complex date arithmetic in queries

- Pre-computed attributes: day_of_week, week_number, quarter, is_weekend, is_holiday
- Supports fiscal year analysis through quarter/year columns
- Holiday flag enables seasonal promotion analysis (Egyptian holidays included)
- Integer date_key (YYYYMMDD format) for efficient joins and range filtering

dim_customer — Customer Attributes

- **Segmentation Fields:** customer_segment (VIP, Regular, New) for targeted marketing
- **Demographics:** age_group, gender enable cohort analysis
- **Geography:** city, region, country for location-based insights
- **Temporal Tracking:** registration_date enables customer lifecycle analysis

dim_product & dim_category

- **Hierarchy:** dim_category contains parent_category for two-level product hierarchy
- **Pricing:** Both unit_price and cost_price stored for margin analysis
- **Lifecycle:** is_active flag filters discontinued products from recommendations
- **Brand Analysis:** brand field enables competitive positioning studies

dim_payment & dim_shipping

- Payment methods (credit card, digital wallet, COD) for payment preference analysis
- Shipping types (express, standard, same-day) with delivery_days for logistics optimization
- Carrier performance tracking through carrier field

3. Dimensional Modeling Approach

3.1 Synthetic Data Generation Strategy

The synthetic dataset was designed to simulate realistic e-commerce patterns while ensuring reproducibility and statistical validity.

Date Dimension (02_seed_dimensions.sql)

- **Approach:** Recursive CTE generates 730 consecutive dates (2 years)
- **Rationale:** 2-year span enables year-over-year comparison and seasonal analysis
- **Holiday Logic:** Egyptian national holidays hard-coded for location-specific analysis

Product Dimension

- **140 Products:** Balanced across 12 subcategories (smartphones, laptops, accessories)
- **Price Realism:** Category-specific price ranges (e.g., laptops: EGP 28k-98k, chargers: EGP 200-1,800)
- **Cost Ratio:** Cost = 62-84% of unit_price (realistic margin range 16-38%)
- **Product Lifecycle:** 5% marked inactive (is_active = FALSE) to test recommendation filtering

Fact Table Transactions (03_seed_fact_order_lines.sql)

- **Volume:** 1,000 order lines (~500 orders, avg 2 lines per order)
- **Distribution:** Price tiers: 25% low (EGP 350-2,350), 50% mid (EGP 1,800-13,800), 25% high (EGP 12k-77k)
- **Discount Modeling:** 50% no discount, 35% small (5-15%), 15% large (15-30%)
- **Random Seeding:** RAND(n * prime) ensures deterministic but varied distribution

3.2 Indexing Strategy

Indexes were strategically placed to optimize common analytical query patterns:

Index	Purpose
idx_fact_date	Accelerates time-series queries (monthly/yearly aggregations)
idx_fact_date_customer	Composite index for customer purchase history analysis
idx_fact_date_product	Optimizes product performance over time queries
idx_fact_order	Enables fast order-level aggregations (AOV, basket analysis)

4. KPI Framework & Business Logic

Six analytical views (05_kpi_views.sql) were created to encapsulate complex business logic. Each KPI was derived without predefined formulas, requiring analytical reasoning about business meaning.

4.1 Monthly KPI Summary (v_monthly_kpis)

Analytical Reasoning:

- **Revenue vs. Profit Distinction:** Revenue = SUM(net_amount) represents top-line sales; Profit = SUM(profit_amount) shows operational efficiency after costs
- **Average Order Value (AOV):** Revenue / COUNT(DISTINCT order_id) — measures transaction size, not customer value
- **Profit Margin %:** (Profit / Revenue) × 100 — normalized metric for cross-period comparison
- **Units Sold:** SUM(quantity) — separate from revenue to detect pricing changes

4.2 Revenue Growth Analysis (v_monthly_revenue_growth)

Window Function Application:

- **LAG(revenue) OVER (ORDER BY year, month):** Accesses previous month's revenue for MoM% calculation
- **Growth Formula:** ((current - previous) / previous) × 100 — handles NULL for first month
- **Business Insight:** Negative growth flags declining performance; >20% growth may indicate seasonality or promotions

4.3 Product Performance Ranking (v_top_products_by_revenue)

Ranking Strategy:

- **DENSE_RANK() vs. ROW_NUMBER():** DENSE_RANK handles ties (same revenue products share rank)
- **Multi-Metric View:** Revenue AND profit both included — high revenue doesn't guarantee profitability
- **Order Count:** Distinguishes breadth (many small orders) from depth (few large orders)

4.4 Customer Lifetime Value (v_customer_lifetime_value)

CLV Calculation Philosophy:

- **Simple CLV:** SUM(net_amount) — historical total, not predictive (no time-value discount)
- **Customer AOV:** CLV / total_orders — detects high-value vs. high-frequency customers

- **Spending Quartiles:** NTILE(4) OVER (ORDER BY revenue DESC) — segments into top 25%, 50%, 75%, bottom 25%
- **Marketing Application:** Q1 customers receive VIP treatment; Q4 customers need reactivation campaigns

4.5 Repeat Purchase Rate (v_repeat_purchase_rate)

Behavioral Metric Rationale:

- **Definition:** (Customers with >1 order / Total customers) × 100
- **Business Significance:** High rate (>60%) indicates strong retention; low rate (<30%) signals churn risk
- **Data Quality Check:** CTE isolates order-counting logic to ensure accuracy

5. Advanced Analytical SQL Techniques

This project extensively leverages window functions to solve complex time-series, ranking, and customer behavior problems. Each technique is applied with specific analytical intent.

5.1 Window Function Categories

Category	Functions Used	Purpose
Ranking	ROW_NUMBER(), DENSE_RANK(), NTILE()	Product rankings, customer segmentation
Offset	LAG(), LEAD()	MoM growth, time-between-purchases
Aggregate	SUM() OVER(), MAX() OVER()	Cumulative revenue, normalization

5.2 Partitioning Strategy

Critical Design Decision: Per-source-product vs. global normalization

Example from Recommendation System:

```
freq_norm = co_purchase_count / MAX(co_purchase_count) OVER (PARTITION BY
source_product_key)
```

Why PARTITION BY source_product_key?

- Each product has different co-purchase patterns (smartphones vs. chargers)
- Global normalization would bias toward high-volume products
- Per-product normalization ensures fair comparison within each product's candidate set

5.3 Handling NULL Values

Defense-in-depth approach to NULL handling:

1. **COALESCE in calculations:** 0.40 * COALESCE(freq_norm, 0) — ensures NULLs don't poison weighted sum
2. **NULLIF in denominators:** SUM(x) / NULLIF(COUNT(y), 0) — prevents division by zero
3. **IS NOT NULL filters:** WHERE subcategory IS NOT NULL — explicit NULL exclusion

6. Product Recommendation System

The recommendation engine (06_recommendation_top4.sql) implements collaborative filtering principles entirely in SQL, demonstrating how machine learning concepts can be translated to database logic.

6.1 Architecture Overview

Five-Stage CTE Pipeline:

4. **order_products:** Distinct (order_id, product_key) pairs with dates
5. **product_pairs:** Co-purchase counts via self-join on order_id
6. **directed_pairs:** UNION ALL to bidirectionalize (A→B and B→A)
7. **scored:** Multi-factor scoring with normalization
8. **ranked:** ROW_NUMBER() PARTITION BY source to get top-4

6.2 Scoring Components (6 Factors)

Factor	Weight	Analytical Reasoning
Co-purchase Frequency	40%	Core collaborative filtering signal. COUNT(*) in product_pairs measures "market basket" association. Per-source normalization prevents popular products from dominating.
Recency	25%	Exponential decay: EXP(-days_since / 90). 90-day half-life means 3-month-old co-purchases score ~37% of today's. Prioritizes trending pairs.
Category Match	10%	Hierarchy-aware: same subcategory = 1.0, same parent_category = 0.5. Balances discovery (cross-category) with relevance (same-category).
Candidate Popularity	10%	COUNT(DISTINCT order_id) per product. Ensures recommendations aren't obscure items — users trust popular products. Global normalization appropriate.
Profit Margin	10%	AVG(profit_amount / net_amount). Business logic: prefer recommending high-margin products. Weak signal (10%) to avoid pure profitability bias.
Availability	5%	Boolean filter (is_active = TRUE) + 5% bonus. Hard constraint: inactive products filtered out completely. Bonus acknowledges in-stock priority.

6.3 Critical Improvements Applied

Six refinements were made to the recommendation algorithm based on analytical debugging:

9. **Per-source normalization:** Changed OVER () to OVER (PARTITION BY source_product_key) for freq_norm and recency_norm
10. **NULL safety:** COALESCE(score, 0) prevents NULL propagation in weighted sum
11. **Category hierarchy:** COALESCE(subcategory, '') for NULL-safe comparison + parent_category join
12. **Exponential decay:** Replaced 1/(1 + days) with EXP(-days/90) for sharper recency bias
13. **Availability filter:** AND is_active = TRUE before scoring (not after)
14. **Weight rebalancing:** 40/25/10/10/10/5 allocation prioritizes behavioral signals over metadata

6.4 Validation Checks

Four validation queries ensure recommendation quality:

- **Coverage check:** Products with 0 recommendations (never co-purchased)
- **Self-reference check:** No source_product_key = candidate_product_key (should return 0 rows)
- **Score distribution:** MIN/AVG/MAX/STDDEV to detect scoring anomalies
- **Diversity check:** Flag if one product appears in >50% of recommendation sets

7. Data Quality & Validation Strategy

The validation script (04_validation_checks.sql) implements a three-tier quality framework:

7.1 Tier 1: Structural Integrity

- **Row Count Verification:** Expected: 730 dates, 120 customers, 140 products, 1000 fact rows
- **Orphan FK Detection:** LEFT JOIN + WHERE dim.key IS NULL → flags referential integrity violations
- **Rationale:** These checks must return 0 issues for the warehouse to be usable

7.2 Tier 2: Arithmetic Consistency

Validation with tolerance for floating-point precision:

- **Net Amount Formula:** ABS(net_amount - (gross_amount - discount_amount)) <= 0.02
- **Profit Formula:** ABS(profit_amount - (net_amount - cost_amount)) <= 0.02
- **Quantity Check:** WHERE quantity <= 0 → should return 0 (no negative/zero orders)

Why 0.02 tolerance?

- DECIMAL(10,2) rounding + multiple arithmetic operations can accumulate 1-2 cent errors
- Business impact of ±0.02 EGP is negligible (< $0.001 USD)

7.3 Tier 3: Analytical Sanity

- **Aggregate Reasonableness:** Total revenue ~EGP 18-35M, AOV ~EGP 35k-65k (realistic for tech e-commerce)
- **Monthly Distribution:** No month should have 0 revenue or >50% of total (would indicate data skew)
- **Time Continuity:** All 24 months (2 years) should have some transactions

8. Implementation Insights

8.1 MySQL vs. SQLite Dual Implementation

The project includes both MySQL DDL (01-06.sql) and SQLite Python implementation (db.py):

Aspect	MySQL Version	SQLite Version
Primary Use	Production teaching/demo	Streamlit dashboard
Data Generation	Recursive CTE	Python loops + Faker
Deployment	Requires MySQL server	In-memory, zero config

8.2 Performance Optimization Decisions

- **View Materialization:** KPI views are NOT materialized — acceptable for 1k rows, would need materialization at 100k+ rows
- **Recommendation CTE Depth:** 5 CTEs deep — readable but could be refactored into temp tables for production
- **Index Selectivity:** Composite indexes (date_key, customer_key) left-prefix rule enables single-column queries

8.3 Code Organization Rationale

- **01_create_dw_schema.sql:** Pure DDL, idempotent (IF NOT EXISTS), self-documenting with comments
- **02-03_seed:** Separated dimensions (02) from fact (03) for incremental loading pattern
- **04_validation:** Standalone validation enables CI/CD integration (exit 1 on failures)
- **05_kpi_views:** DROP VIEW IF EXISTS for safe rerun during development
- **06_recommendation:** Embedded validation queries document expected behavior

9. Dashboard & Visualization

The Streamlit dashboard (app.py) provides interactive exploration of the data warehouse with four specialized pages:

9.1 Overview Page

- 6 KPI cards: Total revenue, profit, orders, customers, AOV, profit margin%
- Time-series chart: Monthly revenue with 3-month moving average overlay
- Growth analysis: MoM% chart with color-coded positive/negative bars
- Cumulative revenue: Line chart showing total revenue accumulation

9.2 Product Analytics Page

- Category revenue pie chart with % share labels
- Top 20 products bar chart with brand color-coding
- Revenue vs. profit scatter plot (detect high-rev/low-profit products)
- Full product rankings table with sortable columns

9.3 Customer Analytics Page

- CLV histogram: Distribution of customer lifetime values
- Segment comparison: VIP vs. Regular vs. New avg revenue
- Spending quartiles: Q1-Q4 breakdown for targeted marketing
- Top 20 customers horizontal bar chart

9.4 Recommendations Page

- **Product selector:** Dropdown with 140 active products
- **Left panel:** Selected product card with price, brand, category, margin
- **Right panel:** Top-4 recommendations in 2x2 grid with scores
- **Scoring explainer:** Collapsible section with weight table

9.5 Design Philosophy

- **Dark theme:** Gradient background (0f0c29 → 1a1a2e → 16213e) for premium feel
- **Color palette:** Purple/blue gradients (#667eea, #a78bfa) for consistency
- **Typography:** Inter font for clean, modern readability
- **Responsive:** clamp() for font-size, st.columns for grid layout

10. Conclusion & Future Enhancements

10.1 Project Achievements

- ✓ Designed production-grade star schema with 6 dimensions + 1 fact table
- ✓ Generated 1,000 realistic synthetic transactions across 2 years
- ✓ Implemented 6 KPI views using advanced window functions
- ✓ Built SQL-driven recommendation engine with 6-factor scoring
- ✓ Created interactive Streamlit dashboard with 4 analysis pages
- ✓ Validated data quality with 3-tier testing framework

10.2 Key Learning Outcomes

15. **Dimensional Modeling:** Grain definition, surrogate keys, slowly-changing dimensions (SCD Type 1)
16. **Window Functions:** LAG/LEAD for time-series, NTILE for segmentation, PARTITION BY for scoped aggregations
17. **Collaborative Filtering:** Co-purchase analysis, recency decay, normalization strategies
18. **Data Validation:** Referential integrity, arithmetic checks, analytical sanity tests
19. **Visualization:** KPI design, color theory, responsive layouts

- 2025" queries

10.3 Final Remarks

This project demonstrates that sophisticated analytical capabilities can be built entirely within SQL. The star schema provides a robust foundation for business intelligence, the KPI views encapsulate complex logic in reusable components, and the recommendation system proves SQL can handle machine learning workloads at moderate scale.

The dual implementation (MySQL for production + SQLite for portability) showcases adaptability, while the Streamlit dashboard makes insights accessible to non-technical stakeholders. Most importantly, every design choice was made with analytical intent — from partition strategies to weight allocation — rather than blindly following templates.

This documentation serves as both technical reference and pedagogical resource, explaining not just what was built, but why each decision was made and what tradeoffs were considered.

— END OF DOCUMENT —